

3 Operator Overloading, Inheritance & Polymorphism

3.1 Concept of Operator Overloading

3.2 Rules for Operator Overloading

3.3 Unary & Binary operator overloading

3.4 Concept of Inheritance

3.5 Types of Inheritance

3.6 Concept of Polymorphism

3.7 Virtual Base Classes

3.8 Pure Virtual functions

Polymorphism in C++ :-

Polymorphism is made up of two words, poly means more than one and morphs means forms. Therefore, polymorphism means the ability to take more than one form. This property makes the same entities such as functions, and operators perform differently in different scenarios. You can perform the same task in different ways. This makes polymorphism in C++ an important OOP concept.

Polymorphism in C++ is using the same functions or operators multiple times in multiple ways.

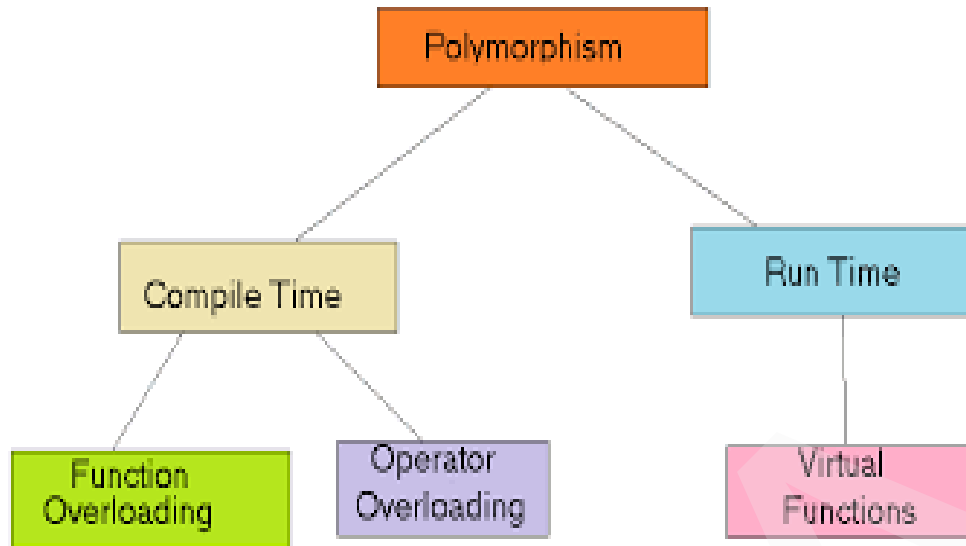
Real-Life Example Of Polymorphism :-

To understand polymorphism, let's take an example from everyday life. Suppose you go to a grocery store and buy different kinds of fruits. All of them are fruits, but each kind has different characteristics. For example, apples are round, oranges are orange, and bananas are yellow. Although all fruits have different characteristics, they can all be grouped under the category of "fruit". This is an example of polymorphism – the ability to group different objects under a single name.

Types of Polymorphism in C++ :-

There are two types of polymorphism in the C++ programming language. These are:

1. Compile time Polymorphism
2. Runtime Polymorphism



1.Compile time Polymorphism :-

When the relationship between the definition of different functions and their function calls, is determined during the compile-time, it is known as compile-time polymorphism. This type of polymorphism is also known as static or early binding polymorphism. All the methods of compile-time polymorphism get called or invoked during the compile time.

Compile-time polymorphism has a much faster execution rate since all the methods that need to be executed are called during compile time.

The compiler performs compile-time polymorphism in C++ in two ways:

- Function Overloading
- Operator Overloading

Operator Overloading in C++ :-

Operator overloading in C++ is basically where different operator functions have the same symbol but different operands. And, depending on the operands, different operator functions are executed. The operands here are the user-defined data types like objects or structures and not the basic data types.

For example, we can make the operator ('+') for string class to concatenate two strings. We know that this is the addition operator whose task is to add two operands. So a single operator '+' when placed between integer operands, adds them and when placed between string operands, concatenates them.

The main difference between normal functions and operator functions is that the name of the operator functions is followed by the keyword “operator” along with the operator symbol that you want to overload.

These operator functions get invoked when their corresponding operators are used in the program.

Syntax :-

```
class className
{
    ... ..
    public
    returnType operator symbol(parameters) {
        ... ..
    }
    ... ..
};
```

Where,

- returnType is the return type of the function.
- operator is a keyword.
- symbol is the operator we want to overload. For example, +, -, *, <, ++, etc.
- parameters are the parameters of the function.

Types of Operator Overloading :-

1. Unary Operator Overloading :-

It is the overloading of an operator operating on a single operand. In the unary operator function, no arguments should be passed. It works only with one class object.

Example of Unary (++) Operator Overloading

```
#include <iostream>
using namespace std;
```

```
class count
{
    private:
        int val;

    public:
```

```
        //Constructor to initialize count to 10
```

```

    count() : val(10) {}

    //Overload ++ when used as prefix
    void operator ++()
    {
        ++val;
    }

    //Overload ++ when used as postfix
    void operator ++(int)
    {
        val++;
    }

    void display()
    {
        cout << "Count: " << val << endl;
    }
};

//Driver code
int main()
{
    count c1;

    //Call the "void operator ++ (int)" function
    c1++;
    c1.display();

    //Call the "void operator ++ ()" function
    ++c1;

    c1.display();
    return 0;
}

```

Output :-
Count : 11
Count : 12

2.Binary operator overloading :-

In the binary operator overloading function, there should be one argument to be passed. It is the overloading of an operator operating on two operands.

Example : Binary (+) Operator Overloading

// C++ program to add two complex numbers by using binary operator overloading

```
#include <iostream>
using namespace std;

class complex
{
    private:
        int real;
        int img;

    public:

        complex(int r=0,int i=0)
        {
            real=r;
            img=i;
        }

        //Overload the + operator
        complex operator +(complex const & obj)
        {
            complex temp;
            temp.real = real + obj.real;
            temp.img = img + obj.img;
            return temp;
        }

        void output()
        {
            cout << "Output Complex number: " << real << "+" << img << "i";
        }
};

//Driver code
int main()
{
    complex c1(12,5), c2(10,2), result;
    result = c1 + c2;
    result.output();

    return 0;
}
```

Output :-
Complex number's addition is : 22+7i

Runtime Polymorphism :-

In runtime polymorphism, the compiler resolves the object at run time and then it decides which function call should be associated with that object. It is also known as dynamic or late binding polymorphism. This type of polymorphism is executed through virtual functions and function overriding. All the methods of runtime polymorphism get invoked during the run time.

Virtual Functions :-

A C++ virtual function is a member function in the base class that you redefine in a derived class.

It is declared using the virtual keyword.

It is used to tell the compiler to perform dynamic linkage or late binding on the function. In late binding function call is resolved during runtime. Therefore the compiler determines the type of object at runtime, and then binds the function call.

There is a necessity to use the single pointer to refer to all the objects of the different classes. So, we create the pointer to the base class that refers to all the derived objects. But, when the base class pointer contains the address of the derived class object, it always executes the base class function. This issue can only be resolved by using the 'virtual' function.

A 'virtual' is a keyword preceding the normal declaration of a function.

When the function is made virtual, C++ determines which function is to be invoked at the runtime based on the type of the object pointed by the base class pointer.

Here we make the pointer of base class and that pointer is pointing to the object of derived class (when the base class pointer contains the address of the derived class object) then we are running any function with the help of pointer then the function of the base class is run only and the derived class function is not running. If we want that our function of the derived class is executed and the base class function should not be executed. This can be achieved by making the virtual function of the base class then and then only the function of the derived class is executed.

Some Key Points About Virtual Functions

1. Virtual functions cannot be static members.
2. Virtual functions are only dynamic
3. They are declared within a base class by using the keyword “virtual”
4. These functions are called during run time
5. Virtual functions are always declared with a base class and overridden in a child class
6. They are accessed through object pointers.
7. They can be a friend of another class.
8. The definition of a virtual function of the base class and all the derived classes must be identical. If the two functions have the same name but different definitions, C++ will consider them as the overloaded functions.

The following program illustrates run-time polymorphism using a virtual function:

```
#include<iostream>
using namespace std;
class A
{
    int x;
    public:
        A(){x=5;}
        virtual void display()
        {
            cout << "Value of x is : " << x<<endl;
        }
};
class B: public A
{
    int y;
    public:
        B()
        {
            y=10;
        }

        void display()
        {
            cout << "Value of y is : " <<y<< endl;
        }
};
```

```
int main()
{
    A *a;
    B b;
    a=&b;
    a->display();
    return 0;
}
```

Output :-

Value of y is : 10

Pure Virtual Function :-

A virtual function is not used for performing any task.

It only serves as a placeholder.

When the function has no definition, such function is known as "do-nothing" function.

The "do-nothing" function is known as a pure virtual function.

A pure virtual function is a function declared in the base class that has no definition relative to the base class.

A class containing the pure virtual function cannot be used to declare the objects of its own, such classes are known as abstract base classes.

Pure virtual function can be defined as:

```
virtual void display() = 0;
```

Example :-

```
#include<iostream>
using namespace std;

class A
{
    int x=5;
    public:
    virtual void display()=0;    //pure virtual function
};

class B: public A
{
    int y=10;
```

```
public:
void display()
{
cout << "Value of y is : " <<y<< endl;
}
};
```

```
int main()
{
A *a;
B b;
a=&b;
a->display();
return 0;
}
```

Output :-

Value of y is : 10

Inheritance in C++ :-

The capability of a class to derive properties and characteristics from another class is called Inheritance. Inheritance is one of the most important feature of Object Oriented Programming.

Advantages of inheritance :-

- By allowing you to inherit common members from a base class into derived classes, inheritance increases code reusability.
- This eliminates the need for duplicating code and reduces the total coding effort.
- Inheritance aids in the hierarchical organization of code by reflecting the links between various classes.
- This improves the readability and maintainability of the code.

Inheritance allows web developers to reuse your code since the derived class or the child class can reuse the members of the base class by inheriting them.

- derived class (child) - the class that inherits from another class
- base class (parent) - the class being inherited from

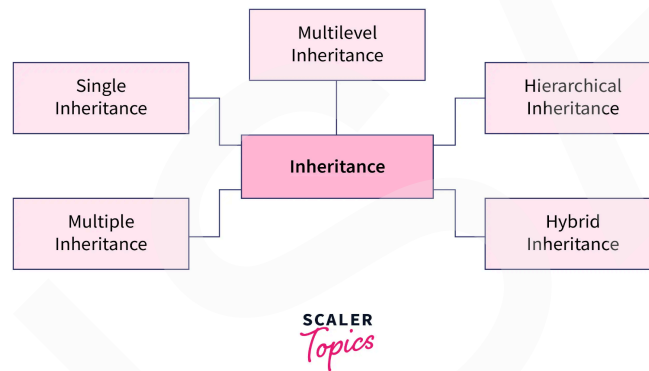
To inherit members of the base class, use the : symbol.

Syntax:

```
class derived_class_name : access_mode base_class_name
{
    //body of subclass
};
```

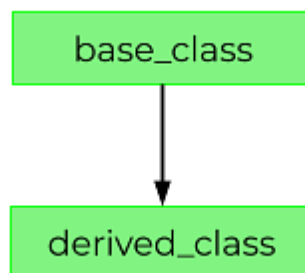
Depending on the way the child class is derived from the parent class or how many parent classes a child class inherits, we have the following types of inheritance in C++, namely:

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance



1. Single Inheritance :-

In this inheritance, a single class inherits the properties of a single base class.(one Base class and one derived class.



Syntax :-

```
class base_class_1
{
    // class definition
};
class derived_class: visibility_mode base_class_1
{
    // class definition
};
```

Example :-

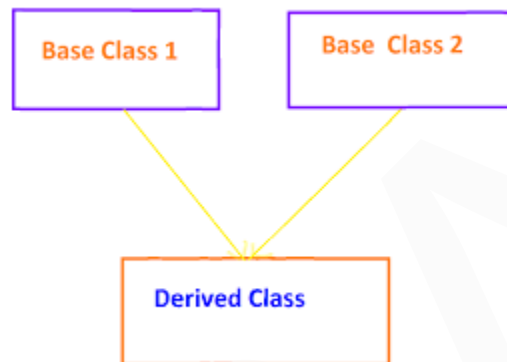
```
#include<iostream>
using namespace std;
class A
{
    public :
    int x=5;
    void display()
    {
        cout<<"x = "<<x;
    }
};

class B : public A
{
    public :
    int y=10;
    void view()
    {
        cout<<"\ny = "<<y;
    }
};

int main()
{
    B b;
    b.display();
    b.view();
    return 0;
}
```

2. Multiple Inheritance :-

When a child/derived class inherits from more than one base/parent class, it is called multiple inheritance.



Syntax :-

```
class A
{
    ....
};
class B
{
    .....
};
class C: visibility_mode_1 A, visibility_mode_2 B
{
    .....
};
```

Example :-

```
#include<iostream>
using namespace std;
class A
{
    public :
    int x=5;
    void display()
    {
        cout<<"x = "<<x;
    }
}
```

```

};

class B
{
    public :
    int y=10;
    void view()
    {
        cout<<"\ny = "<<y;
    }
};

class C : public A,public B
{
    public :
    int z;
    void add()
    {
        z=a+b;
        cout<<"\naddition is= "<<z;
    }
};

int main()
{
    C c1;
    c1.display();
    c1.view();
    c1.add();
    return 0;
}

```

OUTPUT :-

x = 5

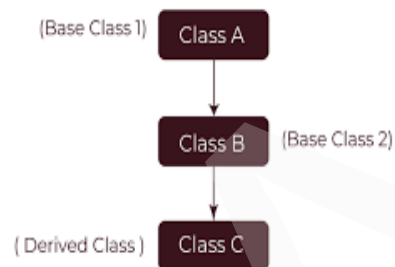
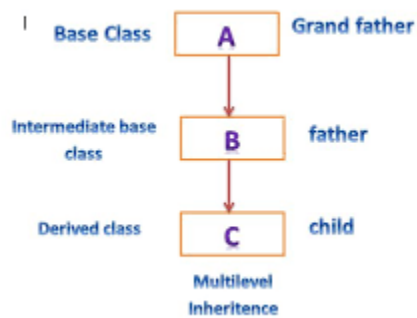
y = 10

Addition is = 15

3. Multilevel Inheritance:

Multilevel inheritance is a process of deriving a class from another derived class.

Suppose there are three classes A, B, and C. A is the base class that derives from class B. So, B is the derived class of A. Now, C is the class that is derived from class B. This makes class B, the base class for class C but is the derived class of class A. This scenario is known as the Multilevel Inheritance.



Syntax :-

```

class A
{
    ...
};
class B: visibility_mode A
{
    .....
};
class C: visibility_mode B
{
    .....
};
    
```

Example :-

```

#include<iostream>

using namespace std;
class A
{
    public :
    int x=5;
    void display()
    {
        cout<<"x = "<<x;
    }
}
    
```

```
};
```

```
class B : public A
{
    public :
    int y=10;
    void view()
    {
        cout<<"\ny = "<<y;
    }
};
```

```
class C : public B
{
    public :
    int z;
    void add()
    {
        z=x+y;
        cout<<"\nz = "<<z;
    }
};
```

```
int main()
{
    C c1;
    c1.display();
    c1.view();
    c1.add();
    return 0;
}
```

Output :-

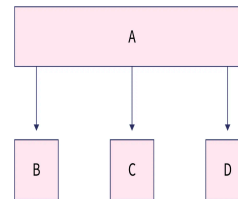
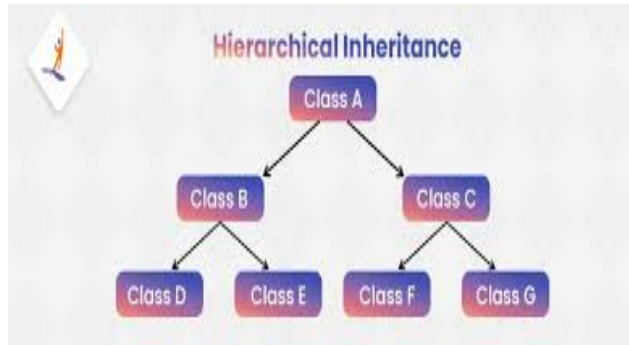
x = 5

y = 10

z = 15

4.Hierarchical Inheritance :-

In hierarchical inheritance, multiple child classes can inherit the property from a single-parent class.



SCALER
Topics

As you can see from the image above, the child classes B, C, and D are derived from the single base class A.

Syntax :-

```
class A
{
    .....
};
class B: visibility_mode A
{
    .....
};
class C: visibility_mode A
{
    .....
};
class D: visibility_mode A
{
    .....
};
```

Example :-

```
#include<iostream>
```

```
using namespace std;
```

```
class A
```

```
{
```

```
    public :
```

```
int x=15,y=10;

void display()
{
cout<<"\nx = "<<x;
cout<<"\ny= "<<y;
}
};

class B : public A
{
    public :
    int z;
    void add()
    {
z=x+y;
cout<<"\nAddition is = "<<z;
    }
};

class C : public A
{
    public :
    int s;
    void sub()
    {
s=x-y;
cout<<"\nSubstraction is = "<<s;
    }
};

int main()
{
B b1;
b1.display();
b1.add();

C c1;
c1.display();
c1.sub();
```

```
return 0;  
}
```

Output :-

x = 15

y= 10

Addition is = 25

x = 15

y= 10

Substraction is = 5